

TOPIK 10

Urban Food Supply Chain Management

Manajemen Rantai Pasok Pangan Kota Yogyakarta

Graph · Circular Queue · Priority Queue · BST · Stack

26
Node

38
Edge

12
Produk

6
Modul

ELT60213 – Algoritma dan Struktur Data

Team Based Project | TA 2025/2026

KELOMPOK 7

Nigel Efendi Sebastian Purba (25051030115)

Kaysan Nawfal Supriyadi (25051030124)

Muhammad Farhan Mustanir (25051030111)

Muhammad Asri Athallah (25051030100)

Teknik Elektro – Universitas Negeri Yogyakarta

Latar Belakang & Tujuan

LATAR BELAKANG

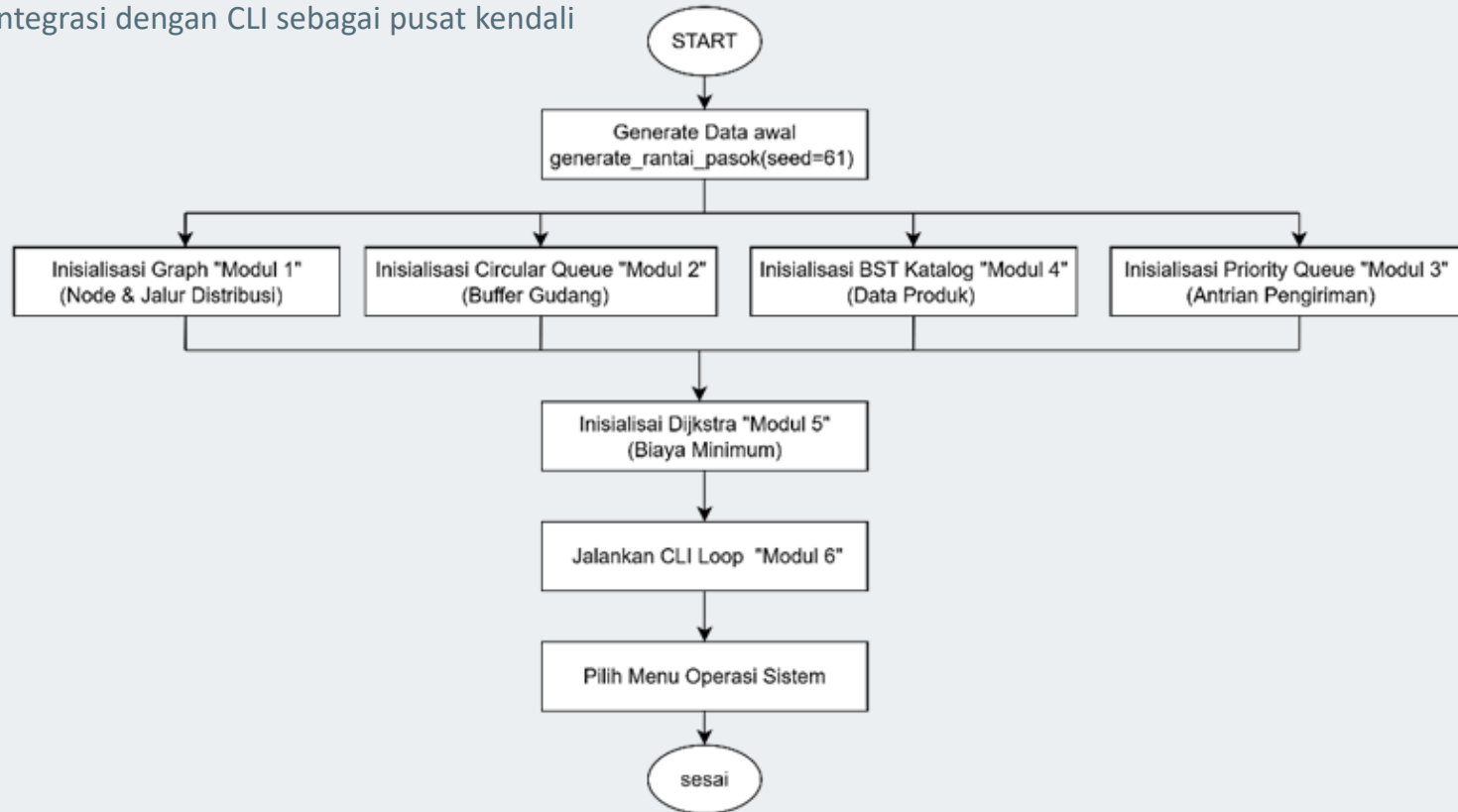
- Ketahanan pangan kota Yogyakarta bergantung pada rantai distribusi dari petani ke konsumen
- Masalah utama: keterlambatan pengiriman, penumpukan stok, & risiko produk kadaluarsa
- Dibutuhkan sistem yang cepat, terstruktur, dan efisien berbasis struktur data

TUJUAN SISTEM

- Membangun jaringan distribusi 26 node berbasis Graph
- Implementasi Circular Queue buffer FIFO per gudang (50 slot)
- Menggunakan Priority Queue untuk pengiriman MENDESAK/REGULER/NORMAL
- Menggunakan BST Katalog untuk insert, search, update & filter kadaluarsa
- Menggunakan algoritma Dijkstra manual untuk rute biaya minimum
- Mem-validasi Big-O dengan benchmark 3 ukuran data

Arsitektur Sistem

6 modul terintegrasi dengan CLI sebagai pusat kendali



Data Flow: generate_rantai_pasok(seed=61) → 26 node • 38 edge • 12 produk • 350 event

Graph Rantai Pasok

Tipe

Adjacency List
Linked List

Node

26
(10P+5D+8M+3G)

Edge

~38
Berbobot

Big-O

 $O(1)$ add
 $O(V+E)$ BFS/DFS

Kegunaan dalam Sistem

- Memodelkan jaringan distribusi pangan kota Yogyakarta
- Petani, distributor, pasar, & gudang sebagai node
- Jalur distribusi sebagai edge berbobot (jarak $\times$ biaya/km)
- BFS untuk audit konektivitas jaringan
- DFS untuk penelusuran urutan kunjungan node

```
// Adjacency List – Tambah Jalur
```

```
class EdgeNode:
    jarak_km, biaya_per_km, next

def
tambah_jalur(u, v, jarak, biaya):
    e = EdgeNode(v, jarak, biaya)
    e.next = self.adj[u] #  $O(1)$ 
    self.adj[u] = e
```

Circular Queue — Buffer Gudang

Kapasitas

**50 Slot
per Node**

Enqueue

**$O(1)$
rear**

Dequeue

**$O(1)$
front**

Prinsip

**FIFO
First In First Out**

Kegunaan dalam Sistem

- Setiap node gudang memiliki 1 buffer CircularQueue (50 slot)
- Produk yang lebih lama masuk akan keluar lebih dulu (FIFO)
- Mencegah penumpukan produk & risiko kadaluarsa
- Circular wrap (modulo) → enqueue/dequeue selalu $O(1)$ tanpa shift
- Memori tetap: cocok untuk sistem kapasitas terkontrol

```
// Circular wrap pointer  $O(1)$ 
```

```
def enqueue(produk):  
    if is_full(): return False  
    buffer[rear] = produk  
    rear = (rear + 1) % kapasitas  
    size += 1
```

```
def dequeue():  
    item = buffer[front]  
    front = (front+1) % kapasitas  
    size -= 1; return item
```

Priority Queue — Antrian Pengiriman

Level 1

MENDESAK

Kadaluarsa
 ≤ 3 hari

Level 2

REGULER

Kadaluarsa
4–7 hari

Level 3

NORMAL

Kadaluarsa
> 7 hari

Kegunaan dalam Sistem

- Linked list terurut — insert pada posisi sesuai prioritas
- Produk MENDESAK selalu di head antrian $\rightarrow$ dequeue $O(1)$
- Mencegah pemborosan akibat produk kadaluarsa sebelum dikirim
- Enqueue $O(n)$ — traversal untuk posisi sisip yang tepat
- Trade-off: insert lambat, ambil cepat (menetapkan head = prioritas tertinggi)

```
// Insertion Sort – Linked List  $O(n)$ 
```

```
def tentukan_prioritas(produk):  
    hari = produk.masa_kadaluarsa_hari  
  
    if hari <= 3: return 1 # MENDESAK  
  
    elif hari <= 7: return 2 # REGULER  
    return  
    3 # NORMAL
```

BST Katalog Produk

Kunci

kode_produk
(string)

Insert/Search

$O(\log n)$
rata-rata

Filter

$O(n)$
inorder traversal

Produk

12 Jenis
Beras, Cabai, dll.

Kegunaan dalam Sistem

- Menyimpan & mencari data 12 produk pangan secara efisien
- Search $O(\log n)$ jauh lebih cepat dari linear $O(n)$
- Insert, update stok, filter produk mendekati kadaluarsa
- Inorder traversal → daftar produk terurut berdasarkan kode
- Basis pengambilan keputusan pengiriman prioritas

```
// BST search & filter kadaluarsa
```

```
def search(node, kode):  
    if node is None: return None  
    if kode == node.produk.kode:  
        return node.produk  
    elif kode < node.produk.kode:  
        return search(node.left, kode)  
    return search(node.right, kode)
```

```
# filter kadaluarsa:  $O(n)$  inorder
```

Dijkstra — Biaya Minimum

Bobot Edge

$\text{jarak_km} \times$
 biaya_per_km

Kompleksitas

$O(V^2 + E)$
tanpa heap

Node/Edge

26 node
38 edge

Output

Rute & Biaya
Termurah

Kegunaan dalam Sistem

- Mencari jalur distribusi berbiaya minimum dari petani ke pasar
- Bobot = $\text{jarak_km} \times \text{biaya_per_km}$ (total ongkos kirim)
- Implementasi manual tanpa library heap (sesuai aturan dosen)
- Rekonstruksi jalur dari array parent secara $O(V)$
- Dijkstra mendominasi >80% runtime — bottleneck komputasi

```
// Dijkstra  $O(V^2+E)$  – Linear scan
```

```
dist[v] = ∞ for all v  
dist[asal] = 0
```

```
for _ in range(V):  
    u = min unvisited node #  $O(V)$   
    mark u as visited
```

```
for v in neighbors(u):  
    w = jarak_km × biaya_per_km
```

```
if dist[u]+w < dist[v]:  
    dist[v]=dist[u]+w  
    parent[v]=u
```

Analisis Big-O & Kompleksitas

Struktur Data	Operasi	Best Case	Worst Case
Circular Queue	enqueue / dequeue	$O(1)$	$O(1)$
Stack	push / pop	$O(1)$	$O(1)$
BST Katalog	search / insert	$O(\log n)$	$O(n)$
BST Katalog	filter_kadaluarsa	$O(n)$	$O(n)$
Priority Queue	dequeue	$O(1)$	$O(1)$
Priority Queue	enqueue	$O(n)$	$O(n)$
Graph	tambah_jalur	$O(1)$	$O(1)$
Graph	BFS / DFS	$O(V+E)$	$O(V+E)$
Dijkstra	cari rute	$O(V^2+E)$	$O(V^2+E)$

Hasil Eksperimen & Benchmark

3 Ukuran Data: S (n=50) • M (n=350) • L (n=1000) • Seed=61 • 5 ulangan/skenario



Temuan Utama

- CQ & Stack: runtime stabil → konfirmasi $O(1)$
- BST: tumbuh log → konfirmasi $O(\log n)$
- Priority Queue: sedikit anomali skala kecil
- Dijkstra S→L: 4.6 ms → 1134 ms (kuadratik)
- Dijkstra >80% total runtime dari 350 event
- Throughput: 67K ev/s (S) → 308 ev/s (L)

⚠ Bottleneck: Dijkstra $O(V^2+E)$ menyumbang >80% total runtime. Optimasi dengan Min-Heap akan mengubah kompleksitas menjadi $O((V+E) \log V)$.

Demo CLI & Keputusan Desain

Terminal — Food Supply Chain CLI

```
>> RUTE_MURAH PTN00 GDG02
Jalur Termurah: PTN00 → PSR02 → PTN02 → GDG02
Total Biaya: Rp 249,278

>> KADALUARSA 7
MENDESAK PRD-002 Tomat 1 hari
MENDESAK PRD-003 Ayam 3 hari
REGULER PRD-000 Beras 7 hari

>>
PROSES_KIRIM
Pengiriman #1 berhasil diproses.
Produk: PRD-002 Tomat x80
```

Keputusan Desain

Graph → Adj. List

Sparse graph: hemat memori $O(V+E)$ vs $O(V^2)$

Circular Queue

Fixed capacity; enqueue/dequeue $O(1)$ tanpa shift

PQ → Linked List

Dequeue $O(1)$; MENDESAK selalu di head

BST → kode produk

Search $O(\log n)$ vs linear $O(n)$ pada list

Dijkstra manual

Sesuai aturan: tanpa library sorting/heap

Perintah CLI:

KIRIM • PROSES_KIRIM • RUTE_MURAH • CEK_STOK • KATALOG • KADALUARSA • BUFFER
• AUDIT_JARINGAN • ANTRIAN • LAPORAN_DISTRIBUSI

Kesimpulan & Saran

Berhasil Dibangun

6 modul terintegrasi, 26 node, 12 produk, 350+ event campuran, CLI interaktif

Validasi Big-O

Teori terkonfirmasi eksperimen: $O(1)$ CQ/Stack, $O(\log n)$ BST, $O(V^2)$ Dijkstra

Insight Utama

Dijkstra = bottleneck >80% runtime.
Pemilihan DS tepat → sistem efektif & terukur

Refleksi Trade-off Desain

- PQ Linked List → dequeue cepat $O(1)$, tapi enqueue lambat $O(n)$ | solusi: Bucket Sort 3 queue
- BST → $O(\log n)$ rata-rata, bisa $O(n)$ jika tidak seimbang | solusi: AVL Tree
- Dijkstra manual → mudah dipahami, tapi $O(V^2)$ | solusi: heap-based $O((V+E) \log V)$
- Circular Queue → $O(1)$ sempurna, tapi kapasitas tetap | solusi: dynamic resize